

Informe Final: Sistema de Gestión Hotelera

Link al repositorio:

<https://github.com/MicaelaCembal/SistemaGestionHotelera-CembalyGryner.git>

Por cuestiones de legibilidad, el UML y los diagramas de secuencia, se encuentran en el repositorio y no en este documento.

Breve descripción del problema

El proyecto consiste en el desarrollo de un sistema integral para la gestión de operaciones hoteleras. La problemática principal radica en la dificultad de coordinar de manera consistente el estado de las habitaciones, las reservas de los huéspedes y la facturación de servicios adicionales. Los objetivos alcanzados fueron los siguientes:

- Automatización del ciclo de vida de la reserva (Pendiente, Confirmada, Finalizada)
- Gestión dinámica de estados de habitación para evitar errores de asignación
- Implementación de un sistema de facturación que integra costos de habitación y servicios consumidos
- Prevención de sobreventa mediante validación de fechas

Listado de requerimientos funcionales resueltos

- Gestión de huéspedes: Registro, búsqueda por DNI y categorización.
- Monitoreo de Habitaciones: Visualización en tiempo real de disponibilidad y estados (Ocupado, Limpieza, etc.).
- Gestión de Reservas: Creación de reservas con cálculo automático de costos basado en el tipo de habitación.
- Control de Overbooking: Bloqueo de reservas si existe solapamiento de fechas.
- Proceso de Check-in: Transición de reserva a estado activo y ocupación de habitación.
- Consumo de Servicios: Carga de servicios (consumo/actividades) a la estadía del huésped.
- Proceso de Check-out: Generación de factura final (Estadía + Servicios) y liberación de habitación a estado de Limpieza.
- Mantenimiento de Habitaciones: Opción para confirmar limpieza y devolver la habitación al estado Disponible.

Justificación del Diseño Arquitectónico

Patrones de Diseño GoF

Patrón State (Comportamiento)

- Problemática: lógica compleja de condicionales if-else para determinar qué acciones permitir sobre una habitación según su estado.
- Solución: Se implementó la interfaz EstadoHabitacion y clases concretas (EstadoDisponible, EstadoOcupado, EstadoLimpieza). La clase Habitacion delega su comportamiento al objeto de estado actual.

- Ventaja: Facilita la extensión (agregar estados como "Mantenimiento") sin modificar la clase Habitación.

Patrón Strategy (Comportamiento)

- Problemática: soportar múltiples formas de pago que podrían cambiar o aumentar en el futuro.
- Solución: Interfaz IMedioPago implementada por PagoTarjeta, PagoEfectivo y PagoTransferencia. La clase Pago utiliza la estrategia seleccionada en tiempo de ejecución.
- Ventaja: Desacopla la lógica del cobro de los detalles técnicos de cada medio de pago.

Patrón Composite (Estructural)

- Problemática: Los servicios podían ser individuales o paquetes promocionales, y debían tratarse uniformemente para el cálculo de precios.
- Solución: Interfaz IServicio como base para ServicioConsumo (hoja) y PaqueteServicios (compuesto).
- Ventaja: Permite al sistema calcular el total de servicios sin importar la jerarquía o agrupación de los mismos.

Patrón Facade (Estructural)

- Problemática: interfaz de usuario (Main) interactuaba con demasiadas clases de modelo y DAOs, generando un alto acoplamiento.
- Solución: Clase ControladorSistema que centraliza todas las operaciones de negocio y coordina las llamadas a los DAOs.
- Ventaja: Ofrece una interfaz simple al cliente y protege la integridad del modelo.

Patrón Observer (Comportamiento)

- Problemática: Diferentes módulos (notificaciones, logs, sistema de alertas) reaccionando ante eventos críticos como el Overbooking o el Check-in.
- Solución: Interfaz IObserver y clase ControladorSistema como sujeto que notifica a los observadores registrados (NotificadorSistema).
- Ventaja: Permite añadir nuevas funcionalidades de respuesta a eventos sin modificar el flujo principal del controlador.

Principios SOLID

- Responsabilidad Única (SRP): Cada DAO se encarga exclusivamente de la persistencia de su entidad, mientras que las clases de estado gestionan solo la lógica de transición.
- Abierto/Cerrado (OCP): El sistema de pagos y servicios está abierto a la extensión (nuevos medios o servicios) pero cerrado a la modificación de la lógica de facturación existente.
- Sustitución de Liskov (LSP): Cualquier implementación de EstadoHabitacion puede ser utilizada por la clase Habitación sin que esta sepa cuál es específicamente, manteniendo la integridad del programa.
- Inversión de Dependencias (DIP): El controlador depende de la abstracción IObserver, y la clase Pago de IMedioPago, no de implementaciones concretas.

Patrones GRASP

- Experto en Información: La clase Reserva es la encargada de calcular su costo base, ya que posee la información de fechas y habitación.

- Controlador: ControladorSistema actúa como el primer objeto más allá de la capa de interfaz que recibe y coordina las operaciones. (Vinculado con Responsabilidad Única/SRP y Facade).
- Polimorfismo: Aplicado en IMedioPago y EstadoHabitacion para manejar comportamientos variables mediante una interfaz común. (Vinculado con LSP y OCP).
- Bajo Acoplamiento: Logrado a través del uso de DAOs y el controlador centralizado, evitando que las clases de modelo dependan de la base de datos o la UI.

Instrucciones de despliegue

1. Clonar el repositorio: git clone
<https://github.com/MicaelaCembal/SistemaGestionHotelera-CembalyGryner.git>
2. Abrir el proyecto en IntelliJ IDEA
3. Agregar el JAR mysql-connector-j-9.7.0 en File. Ahí entras a Project Structure, después a Modules y ahí a Dependencies
4. Tener corriendo o PhpMyAdmin como uso Micaela o MySQL como uso Agustina.
5. Ejecutar Main.java para la interfaz gráfica o MainConsola.java para modo consola
6. La base de datos y tablas se crean automáticamente al iniciar
7. Usuarios por defecto: admin / 1234 (ADMIN) y recep / 4321 (RECEPCIONISTA)

Pruebas de ejecución

Creación de reserva y check-in

```
Seleccione opción: 3
DNI Huésped: 12345678
Número Habitación: 301
Fecha Entrada (Año Mes Día):
2026
06
20
Fecha Salida (Año Mes Día):
2026
06
21

[NOTIFICACIÓN]: RESERVA: Hab 301 para Gomez
>>> Reserva Exitosa. Total Hab: $15000.0
```

```
Seleccione opción: 4
DNI para Check-in: 12345678

[NOTIFICACIÓN]: CHECK-IN: DNI 12345678 ingresó.
>>> Check-in completado.
```

Intento de reserva en una habitación ocupada

```
[SISTEMA DE SEGURIDAD - ALERTA]: OVERBOOKING en Hab 301
>>> Error: Habitación ocupada.
```

Realización de un pago

```
Monto a pagar: 15000
Seleccione Medio: 1.Tarjeta 2.Efectivo 3.Transferencia
2
Estado del pago: PAGADO
```

Carga de servicio

```
-- MENÚ DE SERVICIOS DISPONIBLES --
1. Minibar: Coca Cola [$1200.0]
2. Minibar: Agua Mineral [$800.0]
3. Spa y Masajes [$5500.0]
4. Gimnasio Pase Diario [$2000.0]
5. Desayuno Buffet Extra [$3500.0]
Seleccione ID del servicio: 2
Cantidad: 4

[NOTIFICACIÓN]: SERVICIO: Cargado a DNI 12345678
>>> Servicio cargado.
```

Check-out con costo total

```
[NOTIFICACIÓN]: CHECK-OUT: DNI 12345678. TOTAL: $18200.0 (Hab: $15000.0 + Serv: $3200.0)
>>> Check-out exitoso. Total: $18200.0
```

Autoevaluación

Enfrentamos dificultades iniciales en la sincronización de estados entre la memoria de Java y la persistencia en MySQL. Sin embargo, la implementación de patrones de diseño demostró ser invaluable: el patrón State eliminó la necesidad de condicionales que hacían el código ilegible, y el patrón Facade permitió que el desarrollo de la interfaz de consola fuera sumamente sencillo.

En cuanto al valor de los patrones, proporcionan un lenguaje común y una estructura robusta que facilita el mantenimiento. Como desventaja, la configuración inicial requiere más tiempo y clases, pero este costo se recupera rápidamente al momento de escalar o corregir errores en el sistema.